Progetto di Basi di dati
Università
Ca’Foscari di Venezia

CapyFlights

Progetto ideato e sviluppato da:
Dall’acqua Riccardo

Matricola: 892781

Docente di riferimento: prof. Luca Cosmo

INTRODUZIONE

L'applicazione web creata vuole gestire la prenotazione di voli aerei, dove l'utente ha la possibilità di cercare possibili voli in base alla sua destinazione ed il giorno prefisso, permettendogli anche di selezionare il posto a sedere all'interno dell'aereo.
La compagnia aerea invece ha la possibilità di inserire nuovi voli e di monitorare le statistiche dei propri viaggi aerei.

FUNZIONALITA' PRINCIPALI

L'utente, quando ricerca un volo indicando partenza, destinazione e data, visualizza le principali soluzioni di viaggio disponibili: voli diretti oppure, se presenti, voli con scalo.
I voli mostrati coprono un arco temporale di sette giorni a partire dalla data scelta, così da offrire all'utente la possibilità di visualizzare alternative anche qualora non fossero disponibili voli esattamente nel giorno richiesto.

La lista dei voli trovati mostra le informazioni essenziali:

- codice del volo,
- data e orario di partenza/arrivo,
- durata del volo,
- costo minimo del biglietto.

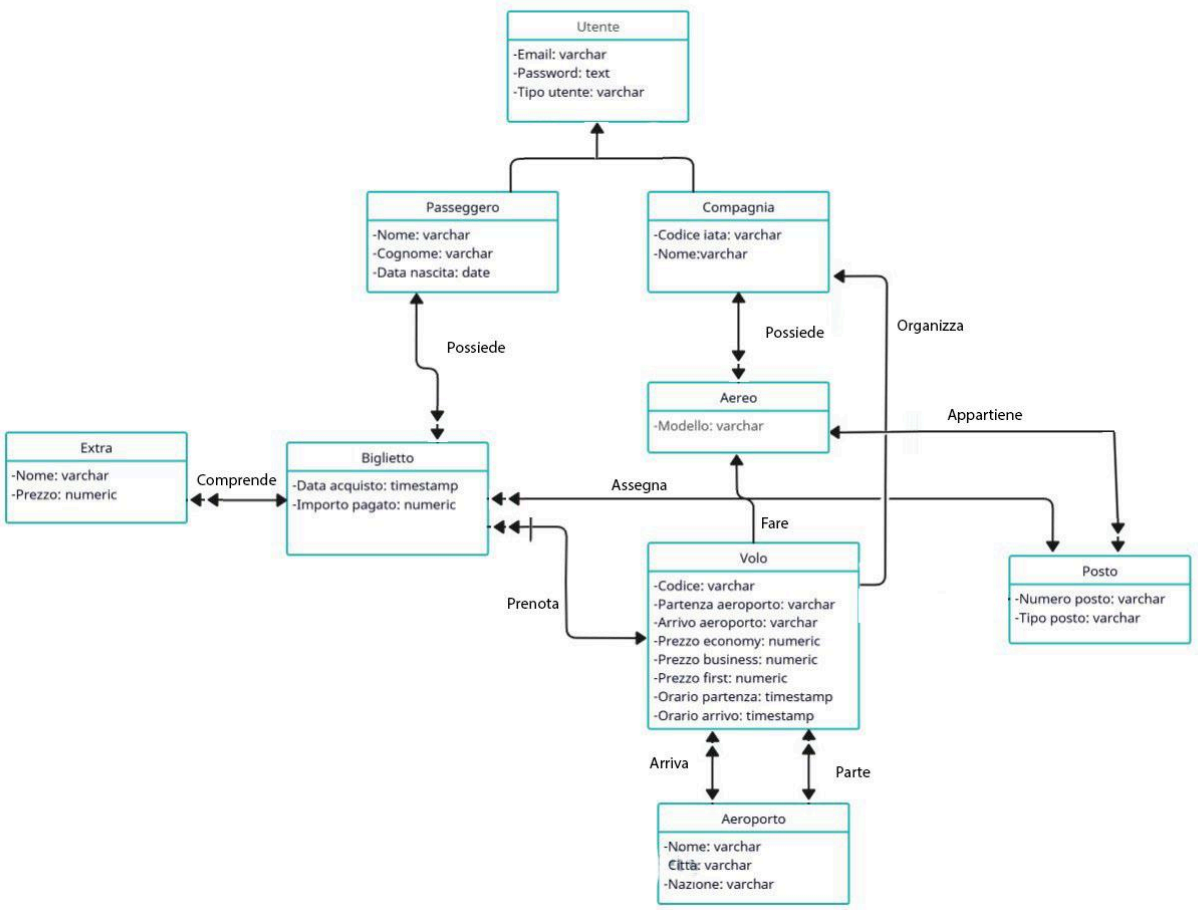
è possibile anche ordinare i voli per prezzo o durata.

Se l'utente è interessato, può accedere ai dettagli del viaggio.
La ricerca e la visualizzazione delle informazioni sono disponibili anche in modalità anonima.
Per procedere alla prenotazione è invece necessario autenticarsi:

1. selezione del posto;
2. eventuali extra di viaggio;
3. conferma con creazione del biglietto nel profilo del passeggero.

Dal lato compagnia aerea:

- creazione e inserimento di nuovi voli,
- accesso a una dashboard con statistiche sui voli (numero di passeggeri, ricavi totali, voli più richiesti).



Relazioni principali

R1 – Appartiene

- **Coinvolge:** Utente → Passeggero, Utente → CompagniaAerea
- **Tipo:** 1:1
- **Descrizione:** Ogni utente è o un passeggero o una compagnia (mai entrambi).

R2 – Prenota

- **Coinvolge:** Passeggero → Biglietto → Volo
- **Tipo:** N:M (un passeggero può prenotare più voli, e un volo può essere prenotato da più passeggeri, tramite l'entità Biglietto)

- **Descrizione:** Rappresenta la prenotazione di un volo da parte di un passeggero, con le informazioni relative al biglietto.

R3 – Organizza

- **Coinvolge:** CompagniaAerea → Volo
- **Tipo:** 1:N
- **Descrizione:** Una compagnia organizza più voli, ma ogni volo appartiene a una sola compagnia.

R4 – Usa

- **Coinvolge:** Volo → Aereo
- **Tipo:** 1:1 (o 1:N se un aereo può essere riutilizzato per più voli in tempi diversi)
- **Descrizione:** Ogni volo è associato a un aereo che lo effettua.

R5 – Decolla/Atterra

- **Coinvolge:** Volo → Aeroporto (partenza/arrivo)
- **Tipo:** 1:N
- **Descrizione:** Ogni volo parte da un aeroporto e arriva in un aeroporto.

R6 – Contiene

- **Coinvolge:** Aereo → Posti
- **Tipo:** 1:N
- **Descrizione:** Ogni aereo contiene un certo numero di posti, suddivisi per tipologia.

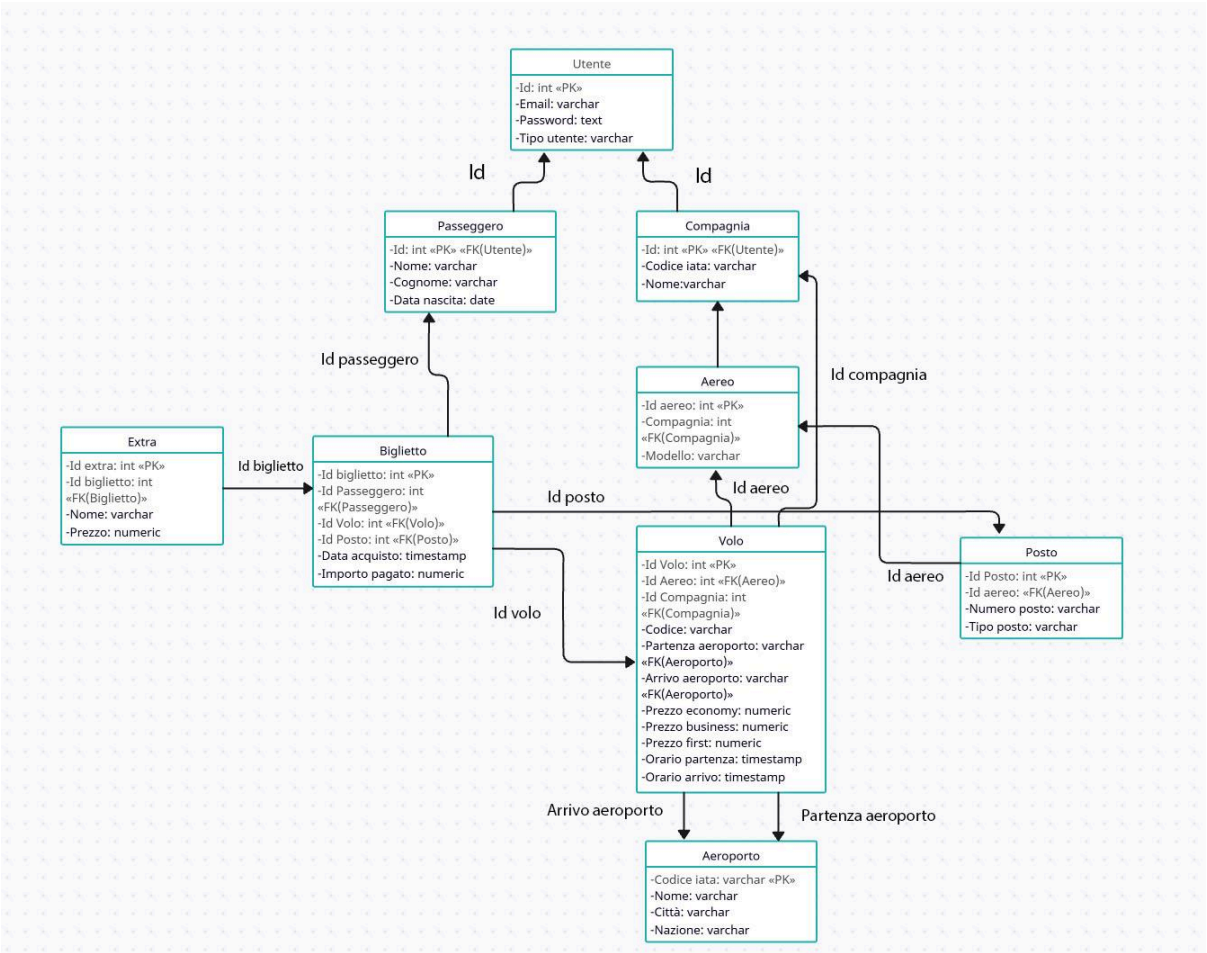
Lo schema a oggetti evidenzia le relazioni tra le classi principali:

- *Utente* come superclasse di *Passeggero* e *CompagniaAerea*.
  Questa scelta consente di gestire in modo uniforme le credenziali di accesso ed evitare duplicazioni di dati.

Nella classe *Volo* è stato inserito l'attributo `id_compagnia`, nonostante potesse essere ricavato tramite la relazione con *Aereo*.
Questa decisione è stata presa per due motivi:

1. un volo può essere gestito da una compagnia diversa da quella proprietaria dell'aereo;

2. migliora l'efficienza delle query sulle compagnie aeree, evitando join non necessari.



PRINCIPALI QUERY

Ricerca di volo con scalo

```
query = db.session.query(
   Volo1, Volo2, Compagnia1.nome, Compagnia2.nome,
   AeroportoP.citta, AeroportoS.citta, AeroportoA.citta
).select_from(Volo1)
.join(Volo2, Volo1.arrivo_aeroporto == Volo2.partenza_aeroporto)
.join(AeroportoP, Volo1.partenza_aeroporto ==
AeroportoP.codice_iata)
.join(AeroportoA, Volo2.arrivo_aeroporto ==
AeroportoA.codice_iata)
```

```
.join(AeroportoS, Volo1.arrivo_aeroporto ==
AeroportoS.codice_iata)
.join(Compagnia1, Volo1.id_compagnia == Compagnia1.id_compagnia)
.join(Compagnia2, Volo2.id_compagnia == Compagnia2.id_compagnia)
.filter(AeroportoP.citta.ilike(f"%{partenza}%"))
.filter(AeroportoA.citta.ilike(f"%{arrivo}%"))
if data_inizio:
   query = query.filter(Volo1.orario_partenza.between(data_inizio,
data_fine))
rows = query.all()
```

```
SELECT v1.*, v2.*
FROM Voli v1
JOIN Voli v2 ON v1.arrivo_aeroporto = v2.partenza_aeroporto
JOIN Aereoporti ap ON v1.partenza_aeroporto = ap.codice_iata
JOIN Aereoporti aa ON v2.arrivo_aeroporto = aa.codice_iata
WHERE ap.citta ILIKE '%' || :partenza || '%'
 AND aa.citta ILIKE '%' || :arrivo || '%'
 AND v1.orario_partenza BETWEEN :data_inizio AND :data_fine;
```

La query permette di trovare un volo con scala confrontando voli che hanno stessa partenza e arrivo dei filtri selezionati ma anche che tra i due voli ci sia un aeroporto in comune così da permettere lo scalo.

Dettagli volo aereo

```
row = db.session.query(
   Voli, CompagniaAerea.nome, Aerei.modello,
   part.nome, part.citta, arr.nome, arr.citta
).join(CompagniaAerea, Voli.id_compagnia ==
CompagniaAerea.id_compagnia)
.join(Aerei, Voli.id_aereo == Aerei.id_aereo)
.join(part, Voli.partenza_aeroporto == part.codice_iata)
.join(arr, Voli.arrivo_aeroporto == arr.codice_iata)
.filter(Voli.id_volo == :id_volo).first()
```

```
SELECT v.*, c.nome AS nome_compagnia, a.modello,
      p.nome AS nome_partenza, p.citta AS citta_partenza,
      r.nome AS nome_arrivo, r.citta AS citta_arrivo
FROM Voli v
JOIN compagnia_aerea c ON v.id_compagnia = c.id_compagnia
JOIN Aerei a ON v.id_aereo = a.id_aereo
JOIN Aereoporti p ON v.partenza_aeroporto = p.codice_iata
JOIN Aereoporti r ON v.arrivo_aeroporto = r.codice_iata
WHERE v.id_volo = :id_volo;
```

Recupera i dettagli del volo selezionato in modo da renderli visibili all'utente.

Conteggio e controllo posti disponibili

```
rows = db.session.query(
   Posti.tipo_posto,
   func.count(Posti.id_posto),
   func.count(Biglietti.id_biglietto)
).join(Aerei, Posti.id_aereo == Aerei.id_aereo)
.outerjoin(Biglietti, and_(Biglietti.id_posto == Posti.id_posto,
Biglietti.id_volo == :id_volo))
.filter(Aerei.id_aereo == :id_aereo)
.group_by(Posti.tipo_posto).all()
```

```
SELECT po.tipo_posto,
      COUNT(po.id_posto) AS totali,
      COUNT(b.id_biglietto) AS occupati
FROM Posti po
JOIN Aerei ae ON po.id_aereo = ae.id_aereo
LEFT JOIN Biglietti b ON b.id_posto = po.id_posto AND b.id_volo =
:id_volo
WHERE ae.id_aereo = :id_aereo
GROUP BY po.tipo_posto;
```

La query permette di contare i posti disponibili presenti ancora nel volo così da poter essere visualizzati nella pagina dei dettagli del volo

Statistiche compagnia aerea

```
voli_totali =
db.session.query(func.count(Voli.id_volo)).filter(Voli.id_compagni
a==:company_id).scalar()
pax_tot =
db.session.query(func.count(Biglietti.id_biglietto)).join(Voli,
Biglietti.id_volo==Voli.id_volo).filter(Voli.id_compagnia==:compan
y_id).scalar()
ricavi =
db.session.query(func.sum(Biglietti.importo_pagato)).join(Voli,
Biglietti.id_volo==Voli.id_volo).filter(Voli.id_compagnia==:compan
y_id).scalar()
```

```
SELECT COUNT(v.id_volo) FROM Voli v WHERE v.id_compagnia =
:company_id;
SELECT COUNT(b.id_biglietto)
FROM Biglietti b JOIN Voli v ON b.id_volo = v.id_volo
WHERE v.id_compagnia = :company_id;
```

```
SELECT COALESCE(SUM(b.importo_pagato), 0)
FROM Biglietti b JOIN Voli v ON b.id_volo = v.id_volo
WHERE v.id_compagnia = :company_id;
```

Vengono contati tutti i voli registrati da una compagnia aerea e le statistiche principali come i biglietti ed i ricavi di essi

PRINCIPALI SCELTE CONCETTUALI

Gestione sessione:
tramite cookie con `session['user_id']` e `session['user_type']`. Logout implementato con `session.clear()`.

Ruoli:
la compagnia aerea può inserire nuovi voli, mentre il passeggero può acquistare biglietti.
Le pagine dei profili sono differenziate:

- il passeggero vede i propri dati e i biglietti acquistati;
- la compagnia vede le statistiche sui voli.

Controlli e integrità:

- disponibilità posti: controllata sia al momento della selezione, sia alla conferma della prenotazione (per evitare conflitti di concorrenza);
- prezzi biglietti ed extra: verificati per essere non negativi;
- biglietti: importo valido e data di acquisto coerente (non successiva all'orario corrente).

Video attesa:

è anche disponibile nella homepage un simpatico video che potrebbe intrattenere l'utente mentre aspetta la risposta di un coetaneo per confermare il volo.

Framework web: *Flask*

ORM: *SQLAlchemy*

Database: *PostgreSQL*,